

MINE-INTEL — Project Context & Handoff Guide

This is the master context document for the **MINE-INTEL** project. Anyone—developer or AI agent—who receives this document should be able to understand the architecture, data flow, map integration, and UI guidelines to safely modify or extend the application without breaking existing functionality.

1. Project Structure & Technology Stack

The project is structured as a modern multi-tier application with a clear separation of concerns:

- `/frontend`: React SPA (Single Page Application).
- `/backend`: Java Spring Boot REST API.
- `/ml-service`: Python AI/ML Microservice.

Technology Stack

- Frontend Framework**: React 19.x with Vite.
- Routing**: React Router DOM (Client-side routing).
- Styling**: TailwindCSS 4.x (Utility-first CSS).
- UI Components**: Lucide React for iconography.
- Mapping**: Leaflet (`leaflet`, `react-leaflet`), `react-leaflet-cluster`, and `leaflet.heat`.
- Backend Framework**: Java 21+ with Spring Boot 3.x (Web, Data JPA).
- Database**: H2 (In-memory/local relational DB).
- ML Service**: Python with FastAPI, Scikit-learn (RandomForest), and SHAP (TreeExplainer).
- Deployment Platform**: Vercel (Frontend), Backend (UNKNOWN — requires confirmation, likely Railway/Render given the stack).

2. The Product

Product Name: MINE-INTEL (Manganese Exploration Intelligence Platform)

One-line description: MINE-INTEL is a geospatial intelligence dashboard that combines geographic data with machine-learning prospectivity scores to identify and prioritize mineral exploration targets.

Product Purpose: It solves the problem of efficiently locating high-value mineral deposits (specifically Manganese) across large geographical regions (e.g., Balaghat, Madhya Pradesh). It is used by Exploration Analysts and Lead Geologists to reduce time spent on manual surveys by focusing field verification efforts on the most statistically probable targets based on satellite, structural, and lithological data.

Core User Workflow: 1. User opens the application and lands on the **Command Center**. 2. User reviews high-level statistics, heat signals, and the regional prospectivity map. 3. User selects a high-priority target (e.g., T-047) from the top targets list or directly on the map. 4. User navigates to the **Prospectivity Explorer** to investigate the specific target in detail. 5. User reviews the AI-generated SHAP analysis, evidence layers, and schedules field verification.

3. Major Screens

Command Center (/)

- Purpose**: The primary dashboard for regional overview and executive intelligence.
- Main Components**: KPI Grid (Candidate Cells, Verified counts), Interactive Leaflet Map (with target clusters and additive heat map), Top Priority Targets list, Intelligence summary cards.
- Data Displayed**: Target clusters, average cluster scores, highest priority ranking.
- Navigation Behavior**: Clicking a top target routes the user to `/explorer?target=T-XXX`.

Prospectivity Explorer (/explorer)

- Purpose**: Detailed geospatial investigation of specific targets.
- Main Components**: Full-screen interactive map, Target detail panel (displaying evidence and counter-evidence).
- Relationship**: The destination for deep dives initiated from the Command Center.

Analyze Area (/analyze)

- Purpose**: Triggers batch ML scoring for the exploration boundary.
- What the user sees**: Currently implemented as an action flow that recalculates or refreshes target prospectivity scores via the backend and ML service.

(Other routes like `/verification`, `/production`, and `/data-health` exist as navigation links but their exact implementations were not fully inspected in this pass.)

4. Map Architecture & Behavior

The mapping system is the **hero component** of the application. It must be treated with extreme caution.

- **Library:** `react-leaflet` wrapped around standard `leaflet`.
- **Base Map:** CartoDB Positron Light (`https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png`).
- **Boundary:** Rendered as a `Polygon` using a predefined array of `[lat, lng]` coordinates (Balaghat region).
- **Heatmap:** Rendered using `HeatmapLayer` which maps target points and `prospectivityScore` as intensity.
- **Clusters:** Handled dynamically by `react-leaflet-cluster`. Groups targets based on proximity and zoom level.
- **Markers:** Rendered as `CircleMarker`. The radius and color are dynamically determined by `prospectivityScore`.
- **Interactions:** Clicking a marker opens a `Leaflet <Popup>` which contains a button to navigate to the target in the Explorer.
- **Data Flow:** The map does not fetch data itself. It receives a `targets` array as a prop or from state and maps over it to render markers.

CRITICAL RULE: Future UI changes should modify the HTML/CSS *around* the map. Do NOT attempt to replace `<MapContainer>`, `<TileLayer>`, or `<MarkerClusterGroup>` with alternative mapping solutions unless explicitly requested.

5. Target Data Model

The core entity is the `ExplorationTarget` (Java Entity) passed down as JSON.

- **targetId** (String): e.g., "T-047". The primary visual identifier.
- **latitude / longitude** (Double): Exact geospatial coordinates.
- **prospectivityScore** (Integer): 0-100 scale representing the likelihood of mineral presence (populated by ML service).
- **priority** (Enum): Categorized as `VERY_HIGH`, `HIGH`, `MEDIUM`, or `LOW`.
- **fieldStatus** (Enum): Status of human verification (`PENDING`, `CONFIRMED`, `NOT_CONFIRMED`, `UNCERTAIN`).
- **featureContributionsJson** (JSON String): Output from the ML SHAP explainer detailing which geological features contributed to the score.
- **explanationText** (String): AI-generated natural language explanation based on SHAP values.

Data Flow: Database (H2) → Spring Boot Repository → `TargetController` → React fetch → React State (`targets` array) → Map Markers / UI Lists.

6. Data Authenticity

- **REAL / CONNECTED:**
 - `prospectivityScore`, SHAP values, and natural language explanations are real outputs from the connected Python `ml-service`.
 - Target Coordinates and IDs are fetched from the live H2 database via the Spring Boot API.
- **MOCK / STATIC:**
 - The exploration boundary polygon (`BALAGHAT_BOUNDARY`) is statically defined in the frontend.
 - The dataset currently represents a simulated/demo region for prototype purposes (as noted by UI badges "SIMULATED DEMO DATA"). However, the *flow* is real.
- **HYBRID:**
 - Some UI aggregate metrics (like "10,000 Candidate Cells") are static placeholders used for visual density, while "High-Priority Targets" and "Field Verified" counts are dynamically calculated using `.filter()` on the real `targets` array.

7. API & Backend Contracts

The backend acts as the orchestrator between the database, the frontend, and the ML microservice.

Base Configuration: Frontend connects to `import.meta.env.VITE_API_BASE_URL` (default: `http://localhost:8080`).

Key Endpoints: - `GET /api/targets`: Returns a JSON array of `ExplorationTarget` objects. - `POST /api/assistant/chat`: Accepts `{ message, targetId, currentPage }` and returns a generated contextual AI response `{ reply }`. - `GET /api/analysis/...` / `POST /api/analysis/...`: Interfaces for triggering ML workflows.

Note: Do not expose `GEMINI_API_KEY` or database credentials in code. Ensure they are loaded via `.env` files.

8. UI/UX Design Language

Layout Philosophy: Map-first. The map takes up the majority of the viewport area, surrounded by compact, high-density analytical panels (dashboards/drawers).
Color Palette: - *Backgrounds:* Slate-50 (`#f8fafc`) for pages, White for cards. - *Primary Accents:* Blue-600 to Blue-700 for primary buttons and active states. - *Borders:* Slate-200. - *Text:* Slate-900 (Headings), Slate-600/500 (Body text). **Typography:** Inter (or system sans-serif font). Headings are bold/extrabold with tight tracking. Small uppercase labels are used extensively for subheadings. **Components:** - *Cards:* White background, 12px or 14px border radius, subtle shadows (shadow-sm). - *Buttons:* Solid colors for primary actions, subtle slate hovers for secondary actions. - *Drawers:* Slide-in panels (e.g., the AI Assistant) that overlay

without breaking the map layout.

9. Priority Visual Language

Priority levels are standardized across the application and dictate badge colors, map marker colors, and heatmap intensity:

- **Very High (Score ≥ 80):** Red (text-red-600, bg-red-50, map marker #dc2626).
- **High (Score 60 - 79):** Orange (text-orange-600, bg-orange-50, map marker #f97316).
- **Medium:** Amber/Yellow (text-amber-600, map marker #facc15).
- **Low:** Green/Slate (map marker #22c55e).

Rule: When adding new visualizations, STRICTLY adhere to this color mapping for consistency.

10. Component Reuse Guide

Before creating new components, check if these exist and reuse them:

- **ChatWidget:** Handles the floating slide-out AI assistant and text-to-speech narration. Do not duplicate AI chat logic.
- **HeatmapLayer:** Extracted Leaflet layer for rendering heat points.
- **MetricCard / InsightCard:** Clean, standardized dashboard cards found in `CommandCenter.tsx`.
- **createProspectivityClusterIcon:** Utility function that calculates and renders the custom HTML `N targets · avg X` badge for clusters.

11. Interaction Contracts

- **Target Row Clicked (List):** Should navigate to `/explorer?target={targetId}`.
- **Map Marker Clicked:** Opens a Leaflet Popup showing a quick summary and a button to investigate the target.
- **AI Chat Context:** The chat widget automatically tracks `location.pathname` and the `targetId` search param to provide the AI with the user's current spatial context. Do not break the URL structure.

12. ■ DO NOT BREAK THESE

1. **The Map Engine:** `react-leaflet` components (`<MapContainer>`, `<TileLayer>`, `<MarkerClusterGroup>`).
2. **Target Data Fetching:** The `fetch('/api/targets')` logic that populates the map.
3. **Cluster Rendering:** The `iconCreateFunction` used by the marker cluster group. Modifying this without care will break the UI cluster badges.
4. **Vercel Routing:** The `vercel.json` file in the frontend must remain intact to prevent SPA 404 errors on reload.
5. **CORS Configuration:** The Spring Boot `@CrossOrigin` annotations are required for the frontend to communicate with the backend.

Visual redesigns should modify presentation (CSS/HTML wrappers), not silently replace underlying data mapping or Leaflet lifecycle hooks.

13. Safe Modification Areas

SAFE TO MODIFY

- CSS classes, layout grid sizes, spacing (Tailwind classes).
- Visual hierarchy, typography, and card placement.
- Adding supporting explanatory UI text or tooltips.

MODIFY WITH CAUTION

- Filtering logic on the `targets` array.
- Adding new Leaflet layers (ensure `zIndex` is managed properly).
- Adding new properties to the `ExplorationTarget` entity (requires DB schema updates).

DO NOT MODIFY WITHOUT EXPLICIT REQUEST

- `main.py` ML inference logic and SHAP extraction.
- Spring Boot Data JPA entity mapping logic.
- `vercel.json` deployment rules.

14. Deployment Context

- **Frontend:** Deployed on Vercel. `npm run build` executes `tsc -b && vite build`.
- **Backend:** Spring Boot executable JAR built via Gradle (`./gradlew bootBuildImage` or `./gradlew build`).
- **ML Service:** Python FastAPI server run via Uvicorn (`python3 main.py` or `uvicorn`).
- **Known Deployment Issues:** The frontend is a React SPA. Without proper rewrite rules in `vercel.json` mapping `/(.*)` to `/index.html`, direct navigation to nested routes (e.g., `/explorer`) will result in a 404 error. This has been resolved and the file must be preserved.

15. Known Issues / Technical Debt

- **Important:** The frontend currently fetches all targets at once (`/api/targets`). If the dataset grows to tens of thousands of targets, this will cause memory and rendering bottlenecks in Leaflet. Pagination or bounding-box-based querying will eventually be required.
- **Minor:** Some UI metrics (e.g., "10,000 Candidate Cells") are hardcoded strings in the dashboard rather than derived calculations.

16. Current Product State

- **CURRENTLY WORKING:** Leaflet map integration, marker clustering, heatmap rendering, dynamic priority calculation, API target fetching, SPA routing, AI Chat drawer with Text-To-Speech.
- **PARTIALLY IMPLEMENTED:** Deep target investigation flows inside the Prospectivity Explorer (UI enhancements planned).
- **PLANNED:** Enhanced visual dashboards for Prospectivity Explorer.

17. Future UI/UX Work Guidelines (For Human & AI Developers)

When creating a new screen or modifying an existing screen:

1. **Inspect existing components:** Do not build a new card or badge if one already exists in `CommandCenter.tsx`.
2. **Inspect existing data flow:** Understand how `targets` state is passed down before inventing new state.
3. **Preserve existing functionality:** The map is sacred.
4. **Create a visual proposal first:** If the change is significant, use placeholder layout tools or generate mockups for approval *before* writing code.
5. **Get design approval.**
6. **Test with real data:** Do not hardcode arrays of fake objects. Use the API.
7. **Verify production deployment:** Ensure Vite builds successfully (`npm run build`) and Vercel routing rules hold up.

18. ■ INSTRUCTIONS FOR FUTURE AI AGENTS

You are working on an existing production-oriented geospatial application.

CRITICAL Directives:

1. Do not assume that a feature is missing simply because it is not immediately visible in the current file. Inspect the existing codebase (`/frontend/src/components` and `/backend/src/main/java`) before creating new components or models.
2. Reuse existing components, specifically Tailwind styling patterns, layout shells (`MainLayout`), and mapping utilities.
3. **Never replace real data with mock data** unless explicitly requested. Always derive UI states (counts, lengths, filters) from the live API responses.
4. **Never replace the existing Leaflet map implementation.** You may add to it (new layers/markers), but do not remove the core engine.
5. Preserve existing APIs and interaction contracts.
6. For significant UI changes, you **MUST** produce a visual proposal before implementation if the user requests one.
7. Do not consider localhost verification sufficient. Always verify that `npm run build` succeeds without TS errors, as the deployment pipeline is strict.

19. File Structure Reference

```
project/
├── frontend/
│   ├── src/
│   │   ├── components/           # Reusable UI (ChatWidget, HeatmapLayer, MainLayout)
│   │   ├── hooks/               # Custom logic (useNarration)
│   │   ├── pages/               # Main screens (CommandCenter, Explorer)
│   │   ├── App.tsx              # Router configuration
│   │   ├── main.tsx             # React mount
│   │   ├── package.json         # Dependencies (Vite, React Leaflet, Tailwind)
│   │   └── vercel.json          # CRITICAL SPA deployment rewrite rules
│   └── backend/
│       ├── src/main/java/com/mineintel/
│       │   ├── controller/      # REST API Endpoints
│       │   ├── model/           # JPA Entities (ExplorationTarget, Evidence)
│       │   ├── repository/      # Data Access (H2 database)
│       │   ├── service/         # Business logic
│       │   └── build.gradle      # Spring Boot config
│       └── ml-service/
│           ├── main.py           # FastAPI entry point
│           ├── train_model.py    # ML training script
│           └── requirements.txt  # Python dependencies
```